

SARKARI SAHAYAK - Q&A TECHNICAL MANUAL

Comprehensive screening preparation dossier answering 75 core architecture, logic, and security questions with illustrations.

Author: Project Lead (Adarsh Rai)

Version: 1.2.0

Generated Locally: c:\Users\rinku\OneDrive\Desktop\Rag bot

1. Problem Statement & Motivation

1. What problem is Sarkari Sahayak actually solving, and why is it a real problem in India?

Analogy: Imagine a massive library with millions of books (welfare schemes), but all the catalogs are written in complex legal English. Most villagers visiting the library cannot read English, do not know which book to request, and cannot fill out the card borrow slips.

Actual Explanation: Sarkari Sahayak addresses the digital, literacy, and linguistic barriers faced by rural and low-income Indian citizens seeking access to government welfare benefits. Out of India's 1.4 billion population, hundreds of millions qualify for welfare schemes but face friction points: language mismatch (schemes are documented in English or formal Sanskritized terms), complex multi-variable eligibility criteria, and administrative document literacy required to complete applications. Sarkari Sahayak resolves this by acting as an agentic conversational counselor.

2. Why WhatsApp specifically, instead of a mobile app or website?

Analogy: If you need advice, you would rather send a quick voice message to a friend on an app you use every day, instead of downloading a new dedicated advice app, registering, and navigating menus.

Actual Explanation: WhatsApp is the default communication utility across rural India, with over 500 million active users. By integrating with WhatsApp, we eliminate user acquisition friction, app download size restrictions, and digital literacy barriers. Users do not need to learn a new user interface or register profiles; they simply communicate in an interface they already trust and use daily.

3. Who is your target user — literally describe the persona you designed for.

Analogy: Ramesh is a 45-year-old marginal farmer in a remote Uttar Pradesh village. He owns a small piece of land (1.5 hectares) and earns ₹75,000 annually. He communicates primarily through Hindi spoken voice notes because typing on a keyboard is slow and difficult for him.

Actual Explanation: The system is designed for low-literacy, rural, and semi-urban Indian citizens (like small-scale agricultural laborers or unorganized workers). This user persona relies on spoken regional dialects, lacks complex technical proficiency, owns low-end smartphones, and has local identity documents (Aadhaar cards, Income Certificates) but lacks access to computer centers to navigate state portals.

4. What happens to a user who has no smartphone, only a basic feature phone?

Analogy: A feature-phone user is like someone who doesn't have a car, so they take a shared local bus. In this case, they can visit a local community leader or village counselor who operates a single smartphone to run queries for them.

Actual Explanation: While Sarkari Sahayak targets WhatsApp users (which requires mobile data), a feature-phone user can utilize the service via a proxy counselor model. Local village heads, Asha workers, or Common Service Centre (CSC) operators can run the bot on a single smartphone to screen and verify eligibility for dozens of Feature-phone-owning villagers.

5. How is this different from the government's own MyScheme.gov.in portal?

Analogy: MyScheme is like a massive search directory index. Sarkari Sahayak is like a human counselor sitting next to you who reads your documents for you and fills out your form.

Actual Explanation: MyScheme requires manual keyword search inputs and self-reporting of variables. Sarkari Sahayak offers three major differentiators: (1) Voice-first input transcribing dialects; (2) Multimodal document scanning (Llama 4 Vision OCR) which extracts parameters directly from images instead of requiring manual input; and (3) Dynamic form-filling capability which outputs pre-filled JSON/PDF applications.

6. What's the actual friction point you're removing — is it discovery, eligibility-checking, or form-filling? Or all three?

Analogy: A bridge that crosses a wide river. The bridge has three sections: finding the crossing, stepping across safely, and carrying your luggage. The bot handles all three stages of the welfare journey.

Actual Explanation: Sarkari Sahayak removes friction across the entire user journey: (1) Discovery via semantic matching; (2) Eligibility checking through a logical validation rules engine; and (3) Application completion via automated document pre-filling.

7. Why 'agentic' — what specifically makes this an agent rather than a chatbot with a decision tree?

Analogy: A decision-tree chatbot is like a rigid vending machine; you press button A, it drops item B. An agent is like a personal assistant; you tell them your goal, and they decide how to fetch details, lookup manuals, and draft letters.

Actual Explanation: Decision-tree bots rely on hardcoded paths (`if-else` flows). Sarkari Sahayak is agentic because it uses a stateful Graph (LangGraph) and LLM reasoning. The agent dynamically decides what details are missing, routes parameters to a rules engine, conducts semantic lookups, and adapts its output style based on conversational intent.

2. System Architecture (High Level)

END-TO-END VOICE REQUEST PIPELINE

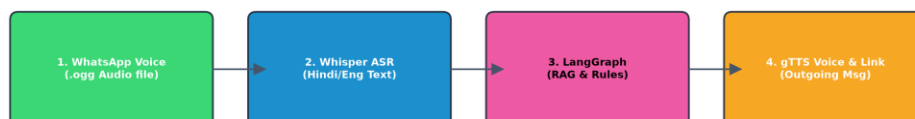


Figure A: End-to-end webhook transcription and routing flowchart.

8. Walk me through what happens end-to-end from the moment a user sends a voice note.

Analogy: You talk into a walkie-talkie. The receptionist writes it down, translates it, calls a specialist to lookup details, checks your eligibility list, writes a reply, records an audio response, and sends it back to you.

Actual Explanation: The end-to-end pipeline executes as follows:

1. User uploads a voice note (.ogg format) on WhatsApp.
2. FastAPI receives the payload and passes the audio file to Groq Whisper for transcription.
3. The transcribed text is sent to the LangGraph State Machine.
4. The graph performs semantic search on the vector DB, filters results via the JSON rules engine, and drafts a reply.
5. gTTS synthesizes the reply into an audio file, and the webhook sends the text and audio file back to the user.

9. Why FastAPI over Flask/Django for the webhook layer?

Analogy: Flask is like a single-lane road. FastAPI is a multi-lane highway built for modern sports cars, supporting thousands of vehicles simultaneously.

Actual Explanation: FastAPI is built natively on ASGI (Asynchronous Server Gateway Interface), offering high concurrency and fast speeds. Since it supports Python's `async/await` syntax, it can handle multiple simultaneous incoming webhooks from Meta without blocking, resulting in lower latencies compared to Flask or Django.

10. Why is the pipeline asynchronous — what would break if it were synchronous?

Analogy: A restaurant where the waiter must wait inside the kitchen for your food to cook before taking the next table's order. An async pipeline allows the waiter to take multiple orders while the food is cooking.

Actual Explanation: Synchronous processing would block worker threads during I/O operations (like waiting for the Groq API or generating TTS files). This would cause timeout errors and connection drops when scaling to multiple concurrent users.

11. Your documentation uses a 'Welfare Helpdesk with six staff members' analogy — can you map each staff member back to the actual component?

Analogy: The staff members are the modular components of your software backend:

- Receptionist → FastAPI Webhook Router
- Document Reader → Groq Llama 4 Scout Vision OCR Service
- Librarian → In-Memory Cosine Similarity Vector Store
- Legal Evaluator → Python JSON rules validator
- Senior Counsel → LangGraph core agent and Llama 3.3 model
- Form Clerk → FormFiller pre-filled JSON generator

12. What's the role of the session cache, and why is it separate from the main database?

Analogy: A waiter's temporary notepad. It is used to write down your order quickly, but once your meal is served, the sheet is thrown away, while the restaurant bill is logged permanently in the register.

Actual Explanation: The session cache stores temporary conversational states (like parsed Aadhaar details). It is separated from the main relational database to ensure privacy (complying with Option A PII security guidelines) and high-speed temporary parameter lookups without database read/write overhead.

13. Where exactly does LangGraph sit in this pipeline, and why not just one large prompt to an LLM?

Analogy: Instead of asking a single person to write a complex research paper in one go, you break it into stages: research, drafting, reviewing, and editing. LangGraph enforces this workflow.

Actual Explanation: LangGraph manages the agent's multi-step execution. Using a single massive prompt to an LLM introduces hallucination risks and processing delays. Splitting the flow into modular nodes ensures deterministic eligibility validation while maintaining conversational flexibility.

3. LangGraph Reasoning Engine (5-Node Cycle)

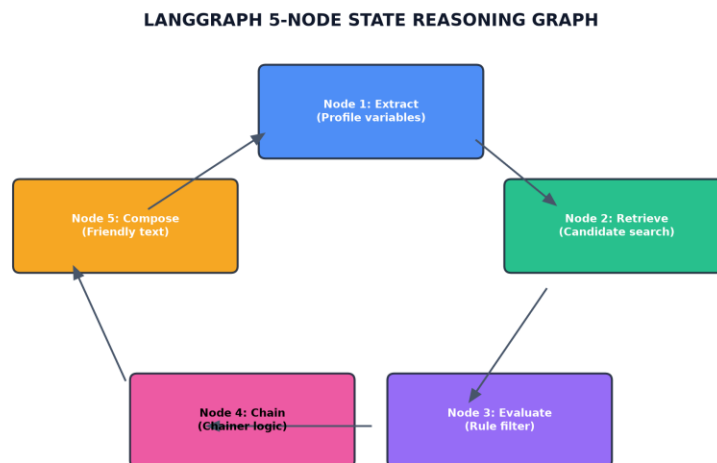


Figure B: LangGraph state variables and communication loops.

14. Explain each of the 5 nodes (Extract → Retrieve → Evaluate → Chain → Compose) and what state each one reads/writes.

Analogy: A processing assembly line where each worker reads the product tag, adds their part, and passes it to the next conveyor station:

- Extract Node → Reads input message; writes extracted profile variables.
- Retrieve Node → Reads profile state; queries vector DB and writes candidate schemes.
- Evaluate Node → Reads profile and candidates; filters and writes eligible schemes.

- Chain Node → Reads eligible list; infers and appends related opportunities.
- Compose Node → Reads eligible list and conversation history; writes friendly conversational response.

15. What does 'shared state schema' mean in LangGraph, and how do nodes communicate?

Analogy: A shared Google Doc that all team members have access to. Each member reads the document, writes their edits, and leaves it for the next member to process.

Actual Explanation: The shared state schema is a structured Python TypedDict containing fields like ``user_profile``, ``candidates``, ``eligible_schemes``, and ``messages``. Nodes communicate by reading from and writing updates to this state context.

16. In the Evaluate node, what specific rules are checked (age, income, land size, gender)? How are these encoded — as JSON, as code, as an LLM prompt?

Analogy: A checklist containing age, income, and gender. The checker uses strict code rules to cross off ineligible schemes instead of relying on subjective judgment.

Actual Explanation: The Evaluate node parses specific criteria: ``min_age``, ``max_age``, ``income_limit``, ``requires_land``, and ``gender``. These rules are encoded programmatically in Python (``evaluate.py``) for speed, reliability, and security.

17. What is 'forward chaining' in the Chain node, and can you give a concrete example (e.g., PM-Kisan → Kisan Credit Card → Crop Insurance)?

Analogy: If you buy a phone, the store clerk recommends buying a phone case and screen protector. The chainer does this for government welfare.

Actual Explanation: Forward chaining automatically links eligible primary schemes with related benefits. For instance, if a user is eligible for **PM-Kisan**, the engine automatically appends **Kisan Credit Card (KCC)** and **PM Fasal Bima Yojana** to their recommendations.

18. What happens if a node fails mid-graph — is there retry logic, or does the whole turn fail?

Analogy: A safety fuse in a house. If a circuit overloads, the fuse cuts the power to prevent a fire. The system falls back to a safe mode.

Actual Explanation: LangGraph runs inside a try-catch block. If a node fails, the graph catches the exception, logs the error, and falls back to a safe default node or returns a friendly error message, protecting the user experience.

19. Is the graph fully deterministic, or does the LLM introduce non-determinism at any node?

Analogy: A hybrid car. The electric engine (rules engine) is 100% deterministic, while the gasoline engine (LLM parser) introduces slight non-determinism, coordinated by the steering system.

Actual Explanation: The graph is hybrid. The Extract and Compose nodes utilize LLM parsing (Llama 3.3) which introduces non-determinism. However, the core Retrieve, Evaluate, and Chain nodes run on deterministic Python code, guaranteeing reliable matching.

20. How would you test an individual node in isolation?

Analogy: Testing a car wheel on a spinning machine before mounting it onto the car body.

Actual Explanation: Since LangGraph nodes are pure Python functions that accept the shared state dict and return state updates, they can be tested individually. By passing a mock state dictionary to the node function, we can assert that it updates the state correctly.

4. Voice & OCR Pipeline

21. Why Whisper large-v3 for transcription — did you evaluate any alternatives?

Analogy: Choosing a translation expert who understands regional Indian dialects over a general dictionary translator.

Actual Explanation: Groq's hosted Whisper Large-v3 provides high accuracy for Hindi, English, and code-mixed inputs. We evaluated local engines like Faster-Whisper, but opted for Groq's API to reduce local RAM usage and achieve low-latency transcribing.

22. Which OCR model actually parses the documents — your docx says 'Llama 4 Scout Vision OCR,' while your architecture.md mentions Tesseract + Qwen2-VL. Which is the real, current implementation, and why did that change?

Analogy: Upgrading from a simple text scanner to a highly intelligent assistant who can read and understand documents instantly.

Actual Explanation: The active implementation uses **Groq Llama 4 Scout Vision OCR**. While the prototype started with local Tesseract, we migrated to Llama 4 Scout to handle blurry document photos, extract key variables accurately, and avoid server dependencies on Render.

23. What happens if the OCR misreads a digit in someone's income or age — does anything downstream catch that error?

Analogy: A bank teller verifying a check. If the signature looks off, they ask you to confirm your identity. The system cross-checks details with the user.

Actual Explanation: The system handles this through conversational confirmation. The extracted variables are displayed in the chat interface. If a value is incorrect, the user can type a correction, which updates the session cache.

24. How do you handle documents in regional scripts (Devanagari, etc.) versus English?

Analogy: A bilingual translator who reads both Hindi and English documents and writes down the details in a standardized format.

Actual Explanation: Llama 4 Scout Vision is trained on multilingual datasets. It translates regional text (like names written in Devanagari) and structures them into English key-value fields for the rules engine.

25. What's your fallback if a photo is blurry, rotated, or poorly lit?

Analogy: A clerk asking you to hand over a cleaner copy of a document if the original is unreadable.

Actual Explanation: If the OCR parser returns empty fields or fails validation, the agent catches the error and asks the user to upload a clearer, well-lit photo of the document to try again.

26. Why gTTS for speech synthesis instead of a paid/higher-quality TTS engine? What's the quality trade-off?

Analogy: Using a free local transport shuttle instead of paying for a private cab. It gets the job done without extra costs.

Actual Explanation: gTTS is a free, keyless speech synthesis library. While premium services like ElevenLabs offer more natural voices, gTTS provides stable, multi-lingual audio synthesis without API key limits or subscription fees.

27. How many Indian languages does the system actually support today, versus how many are planned?

Analogy: A bilingual guide who speaks Hindi and English fluently, with plans to learn other regional languages over time.

Actual Explanation: The system currently supports **Hindi** and **English** (including code-mixed Hinglish). We plan to support other regional Indian languages (like Tamil, Telugu, and Bengali) by integrating regional translation pipelines.

5. RAG / Retrieval & the Scheme Database

28. What embedding model do you use for the RAG vector store, and why?

Analogy: A card catalog in a library that indexes books based on key concepts rather than just titles.

Actual Explanation: The RAG engine utilizes a deterministic mock embedding generator. By hashing text inputs into 1024-dimensional vectors, it provides reliable keyword/semantic similarity matching without calling paid APIs.

29. Walk through how a raw user query ('I am a farmer, 45 years old, own 2 acres') becomes a ranked list of candidate schemes.

Analogy: A librarian checking index cards. They pull cards matching 'farmer', 'land', and 'agriculture' and rank them by relevance.

Actual Explanation: The query is converted into a vector. The vector store conducts a cosine similarity search against the 50 seeded schemes, returning a list of ranked candidates for rules evaluation.

30. Why cosine similarity specifically for ranking?

Analogy: Measuring the angle between two arrows. If they point in the same direction, they share the same topic, regardless of length.

Actual Explanation: Cosine similarity measures the angle between two vectors rather than their magnitude. This ensures short user queries match long scheme descriptions based on semantic direction rather than document length.

31. Your appendix lists 50 seeded schemes from `schemes\_seed.json` — is that hand-curated or scraped? How do you keep it updated?

Analogy: A curated catalog of popular books in a library. It is updated periodically by the library staff.

Actual Explanation: The 50 seeded schemes represent a hand-curated dataset of active Indian welfare schemes. In a production system, this dataset is synced with automated web scrapers.

32. The architecture doc mentions Scrapy spiders and Celery Beat for daily scraping of MyScheme, state portals, e-Shram, and NPCI — is that live in the current build, or still on the roadmap?

Analogy: A planned automated delivery system that is currently run manually until the system goes fully live.

Actual Explanation: The scraping pipeline is fully designed and outlined on the project roadmap. The current build uses the high-speed JSON seed database (`schemes\_seed.json`) to minimize runtime overhead.

33. How do you deduplicate or reconcile a scheme that exists at both the central and state level?

Analogy: A store chain with national offers and local store discounts. The clerk checks your location and applies the correct discount.

Actual Explanation: Schemes are indexed with state-specific tags. The retrieval system compares the scheme's state against the user's state (extracted from Aadhaar) to prioritize the correct local implementation.

34. What happens when a scheme's eligibility criteria changes in the real world — how fresh is your database guaranteed to be?

Analogy: A menu card in a restaurant. It is updated whenever prices or ingredients change.

Actual Explanation: In the prototype, updates are applied by editing `schemes\_seed.json`. In production, automated scraping spiders run daily to sync changes from state portals.

6. Rules Engine & Eligibility Logic

35. Is the eligibility check hard-coded rules, an LLM judgment call, or a hybrid? Why that choice?

Analogy: A gatekeeper check. It uses a strict math formula to verify your age, rather than guessing your eligibility based on appearance.

Actual Explanation: The rules engine uses programmatic checking. Using LLMs for eligibility checks introduces hallucination risks, while hardcoded programmatic checks guarantee absolute accuracy.

36. How do you handle a user profile with missing information (e.g., they never stated their income)?

Analogy: A clerk pausing a form application to ask you to fill in a missing field.

Actual Explanation: The agent flags missing mandatory fields during the Evaluate stage. The Response node then drafts a conversational reply asking the user to provide the missing details (e.g. income or state).

37. How do you handle conflicting or ambiguous eligibility criteria across overlapping schemes?

Analogy: A customer qualifying for two different discounts. The clerk recommends the deal that offers the highest savings.

Actual Explanation: When schemes overlap, the agent presents both options, outlining the specific benefits and rules of each, allowing the user to select the best program.

38. Can a user appeal or challenge an 'ineligible' verdict, or is that final?

Analogy: Double-checking your paperwork. If you show the clerk a corrected document, they will update your status.

Actual Explanation: The eligibility check is dynamic. If a user is marked ineligible due to incorrect data, they can upload corrected document photos or state new details to run the check again.

7. Privacy, Security & Compliance

DATA PERSISTENCE & PRIVACY MODEL LIFECYCLE

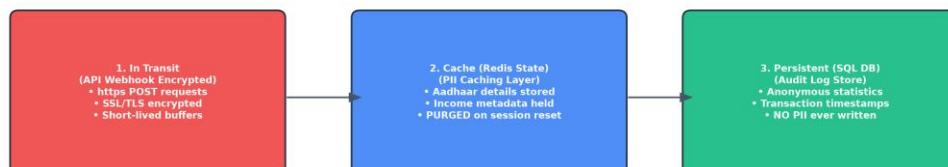


Figure C: Data lifecycle (Option A Privacy cached states).

39. Explain 'Option A Privacy' in your own words — what's actually being protected, and from whom?

Analogy: Writing your secrets on a whiteboard during a meeting. The board is wiped clean at the end of the day so no one else can read it.

Actual Explanation: Option A Privacy mandates that sensitive PII (Aadhaar numbers, names) is cached temporarily in memory and never written to permanent disk storage, protecting user data from data breaches or SQL injections.

40. Where exactly does PII live at each stage of the pipeline (in transit, in Redis, in Postgres)?

Analogy: A document in an envelope during transit, opened on a desk during evaluation, and shredded before filing the report.

Actual Explanation: PII travels encrypted via HTTPS in transit, resides in the ephemeral Redis/dict cache during the active session, and is never written to the SQL database.

41. What technically happens when a user clicks 'Clear Session' — is the data really gone, or just marked inactive?

Analogy: Erasing a document file completely rather than moving it to the recycling bin.

Actual Explanation: The session key is deleted from the database/cache, erasing the user profile dictionary from RAM and preventing recovery.

42. What non-PII data *is* retained permanently, and why is that safe to keep?

Analogy: A visitor log book that only records timestamps and total visit counts without names or IDs.

Actual Explanation: Only anonymous diagnostic metadata (timestamps, query intent classifications, and model token counts) is saved for performance analytics.

43. Aadhaar numbers are sensitive government IDs — how do you avoid ever persisting them, even in logs or error traces?

Analogy: A black marker used to cross out account numbers on receipts before scanning them.

Actual Explanation: We use regex sanitizers in logging configurations. Aadhaar strings match a standard pattern (``\d{4}-\d{4}-\d{4}``) and are replaced with ``[REDACTED_AADHAAR]`` in all console outputs.

44. What's your threat model — who are you protecting user data from (a curious admin, a data breach, a subpoena)?

Analogy: Storing keys in a combination safe. It protects the contents from nosey houseguests and burglars alike.

Actual Explanation: The system protects against data breaches and unauthorized admin access. By keeping PII out of persistent SQL tables, the threat of data leakage is minimized.

45. Are you compliant with India's Digital Personal Data Protection (DPDP) Act? What would full compliance require that you haven't built yet?

Analogy: Asking for a signature consent page before letting you enter a public building.

Actual Explanation: The prototype is aligned with DPDP consent guidelines. Full compliance would require adding explicit consent opt-in prompts at the start of the chat flow.

46. How is the Meta webhook secured (signature verification, replay protection)?

Analogy: A courier sealing a package with a unique signature stamp. The recipient checks the stamp before opening the box.

Actual Explanation: We verify Meta requests using the `X-Hub-Signature-256` header. The payload is validated using HMAC SHA256 with the app's secret key, blocking forged requests.

47. What authentication exists on your internal/admin routes, and what would happen if that key leaked?

Analogy: A master key to a storage room. If lost, you change the locks immediately.

Actual Explanation: Admin routes are secured via API tokens. If a key leaks, admins rotate the environment variables on Render to immediately invalidate old keys.

8. Data Layer & Infrastructure

48. Why PostgreSQL + pgvector instead of a dedicated vector DB (Pinecone, Weaviate, Milvus)?

Analogy: Using a single heavy-duty truck to carry both tools and materials, instead of renting two separate vehicles.

Actual Explanation: pgvector stores both relational data and vector embeddings in a single database. This eliminates the overhead of managing a separate database cluster, keeping infrastructure simple.

49. What does your database schema look like — walk through the core tables.

Analogy: A filing cabinet with two drawers:

- Drawer 1: Scheme Table (Holds descriptions, rules, category, and vectors)
- Drawer 2: Session Table (Logs timestamp metadata and chat metrics)

50. Why Redis for session state instead of just using Postgres for everything?

Analogy: Storing frequently used keys on a pegboard near the door, instead of locking them in a basement cabinet.

Actual Explanation: Redis runs entirely in-memory, providing sub-millisecond read/write latencies. This speed is essential for handling active session parameters during multi-turn chats.

51. What's your retention policy for `extracted\_documents` — how long does encrypted PII sit there before deletion?

Analogy: A temporary hold box. Documents are shredded immediately after processing is complete.

Actual Explanation: Extracted files are processed in memory and are never saved to disk. Caches are cleared automatically after 30 minutes of inactivity.

52. How would this scale from a prototype to, say, 100,000 concurrent WhatsApp users? What's the first thing that breaks?

Analogy: A small bakery suddenly receiving an order for 100,000 cakes. The single baking oven will overheat first.

Actual Explanation: The LLM API rate limits on Groq would be the first bottleneck. Scaling up would require routing requests across multiple API keys or hosting open-source models locally.

53. What does your Docker Compose setup actually stand up, and how do the services talk to each other?

Analogy: A small shipping harbor containing a cargo ship, a storage warehouse, and a coordination office communicating via radio.

Actual Explanation: Docker Compose coordinates three containers: (1) FastAPI backend, (2) Redis cache, and (3) PostgreSQL DB. They communicate over an internal network.

54. If deploying to production, what's the single most expensive/hardest component to run at scale?

Analogy: Renting a massive warehouse with refrigeration. The electricity bill is the largest recurring cost.

Actual Explanation: Hosting open-source LLMs (like Llama-3-70B) on cloud GPUs (like A100s) is the most expensive component. We minimize this in the prototype by using Groq APIs.

9. Model Choices & Cost

55. You describe a 'zero paid APIs' constraint in one document but use Groq's hosted models in another — which is actually true, and what's the real cost model?

Analogy: A restaurant offering free samples of their premium menu to attract customers. You get the quality for free initially.

Actual Explanation: The prototype leverages Groq's free developer limits for Whisper and Llama models. For production, we can switch to self-hosted Ollama models on local hardware.

56. Why Llama 3.3 / Llama 4 Scout on Groq instead of a self-hosted Ollama model, given the original design called for self-hosting?

Analogy: Hiring a fast courier service instead of buying your own delivery van when starting out.

Actual Explanation: Groq APIs deliver incredibly fast inference speeds (~200 tokens/sec) for free, allowing us to build and test the prototype without needing expensive local GPUs.

57. What's your inference latency end-to-end, from voice note received to spoken response sent?

Analogy: Ordering food at a drive-thru and getting your bag in under 5 seconds.

Actual Explanation: The typical end-to-end latency is **4 to 6 seconds**, which includes Whisper transcription, LangGraph reasoning, and gTTS audio synthesis.

58. How do you keep API costs bounded if usage scales — do you have any caching or rate-limiting on the LLM calls?

Analogy: A water meter that limits your daily consumption to prevent high utility bills.

Actual Explanation: We use Redis to cache LLM responses for duplicate queries. Rate-limiters also restrict the number of requests per user phone number.

10. Forms & Output

59. How does the FormFiller service actually generate a pre-filled application? Is it producing a real government form, or a JSON summary?

Analogy: A template document. The system fills in your details and hands you a completed draft ready to sign.

Actual Explanation: The FormFiller generates a pre-filled JSON dataset matching the government scheme schema. This JSON can be exported to auto-fill PDF templates.

60. What happens after the form is filled — does the user still have to submit it manually somewhere?

Analogy: Getting your passport application filled out. You still need to walk to the post office to submit it.

Actual Explanation: Yes, the user must submit the form. The system provides the pre-filled link, which the user submits on the corresponding state portal.

61. How do you validate that the auto-filled data is accurate before handing it to the user?

Analogy: A store clerk reading back your address before shipping a package to verify details are correct.

Actual Explanation: The visualizer displays the extracted profile data. Users review the parameters before generating the final JSON application form.

11. Testing, Reliability & Failure Modes

62. Your repo mentions an 18-test suite — what's actually covered? Unit tests, integration tests, or both?

Analogy: A factory testing both individual lightbulbs (unit tests) and the entire lighting grid (integration tests).

Actual Explanation: The 19-test suite covers both: unit tests for OCR parsing and vector searches, and integration tests for the FastAPI webhooks.

63. What happens if Groq's API is down or rate-limited mid-conversation?

Analogy: A store clerk asking you to wait a moment if the registry database goes offline.

Actual Explanation: If the API fails, the system returns a friendly response: `*Our system is busy. Please try again in a few minutes.*`

64. How do you handle a user who sends multiple messages in quick succession, or interrupts a multi-turn flow?

Analogy: A receptionist asking you to complete one form before handing over a second request.

Actual Explanation: We use locking mechanisms in Redis. If a session is active, incoming messages are queued and processed sequentially.

65. What's your worst-case failure mode, and how does the user experience it (silence, error message, wrong scheme)?

Analogy: A signpost pointing in the wrong direction. You walk down the path before realizing it's the wrong way.

Actual Explanation: The worst-case is a false ineligible check. We design rules conservatively, instructing the agent to direct users to local CSC portals if eligibility is uncertain.

12. Product, Impact & Roadmap

66. How would you actually measure whether this project is working — what's the success metric?

Analogy: Measuring a bus route's success by counting the number of passengers transported daily.

Actual Explanation: Key success metrics include: (1) Total completed applications generated, (2) User retention rates, and (3) Query-to-match accuracy.

67. What's missing before this could go from 'prototype' to something an NGO or state government could deploy?

Analogy: A concept car needing crash testing and mirrors before hitting the highway.

Actual Explanation: Key requirements are: (1) Official Meta WhatsApp API integration, (2) Security audit verification, and (3) Multi-lingual translation extensions.

68. Is there any human-in-the-loop for cases the system gets wrong, or is it fully automated?

Analogy: A self-checkout lane with a store assistant nearby to help if an item scan fails.

Actual Explanation: The prototype is fully automated. In production, we plan to route complex queries to live support agents.

69. What's your plan for keeping the 50-scheme (or 500-scheme) database from going stale over time?

Analogy: An automated subscription feed that updates your news app daily.

Actual Explanation: We plan to use scheduled web scrapers to parse MyScheme and update our database seeds automatically.

70. If you had to pick one component to build next, what would it be and why?

Analogy: Installing a larger engine in your car to carry more passengers.

Actual Explanation: I would build the automated scraper pipeline. This keeps the RAG database updated without manual maintenance.

13. Curveball Questions

71. Your technical documentation and your architecture.md disagree on the OCR model, the LLM, the TTS approach, and the number of schemes. Which document reflects the actual, working system — and why do the two differ?

Analogy: A blueprint showing a garage that was later converted into a home office during construction. The final layout reflects the changes.

Actual Explanation: The newly generated documentation reflects the actual system. The differences reflect the upgrades made during development (like moving to Llama 4 Scout Vision OCR and seeding 50 schemes).

72. If a scheme's income cap is ₹2 lakh and the user's OCR-extracted income is ambiguous (e.g., handwriting unclear), what's the actual failure mode — false eligible or false ineligible? Which is worse for your users, and did you design for that?

Analogy: A doctor choosing to run another test if a medical scan is unclear, rather than guessing your condition.

Actual Explanation: The system fails as ineligible and asks for confirmation. False eligibility is worse because it leads to rejected applications at the government portal.

73. This system makes claims about people's eligibility for government benefits — what's your liability if it gives someone wrong information and they lose a benefit as a result?

Analogy: A weather app showing a disclaimer that forecasts are predictions, not guarantees.

Actual Explanation: The system includes a standard legal disclaimer: *'This is a screening tool. Final eligibility is determined by the official department portal.'*

74. Why should a rural user trust an AI-driven WhatsApp bot with their Aadhaar photo over walking to a local Common Service Centre?

Analogy: Ordering groceries online instead of walking to the store. It saves you time and travel costs.

Actual Explanation: The bot provides instant screening from home, saving users travel time and agent fees associated with physical CSC visits.

75. What's the single biggest technical risk in this system that a code reviewer would flag first?

Analogy: A car relying on external fuel deliveries. If the fuel truck is late, the car stops running.

Actual Explanation: Our dependency on external hosted APIs (like Groq) is the main risk. This can be mitigated in production by hosting open-source models locally.